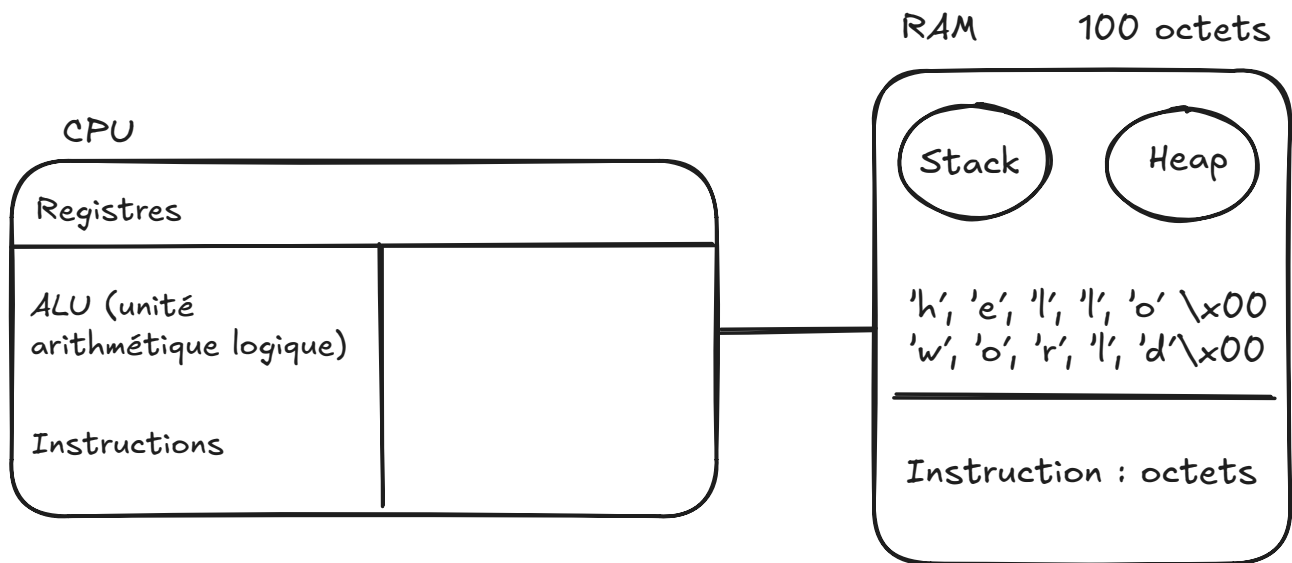


03-03-2025

Architecture matériel : un ordi c'est fait comment ?

- Avec des 0 et 1
- Processeurs



1 octet = 8 bits

$(00000000)_2 = (0)_{10}$

ASCII $(11111111)_2 = (255)_{10}$

`man ascii` → pour voir la table ascii sur Linux

decimal	binaire	hexa	char
48	0x30	60	0
25	0x41		'A'
	0x61		'a'
	0x7F		Del

De 'A' jusqu'à le Space → c'est les caractères de contrôle

Pourquoi on va beaucoup utiliser l'hexa ?

- pratique → conversion de base 2 à base 16 simple
- un octet en hexa prend 1 caractère

decimal	binaire	hexa
0	00000000	00
255	11111111	FF

8 bits → 2 chiffres en hexa

'a' → 01100001

'A' → 01000001

Il y a seulement un chiffre de différence entre les lettres majuscules avec les minuscules

A > a

RAM et encodage

Dans la RAM, des bits sont stockées, mais des fois nous voulons stocké des chiffres et des mots et donc → ASCII (en réalité UTF-8 est utilisé pour encoder les caractères)

UTF-8 est compatible avec ASCII sur les 128 premiers caractères

127 (ASCII) → bit de gauche (fort) est 0

et donc pour UTF-8, si le bit de gauche est 0, c'est du ASCII mais si c'est un 1 c'est de UTF-8

UTF-8 → 110_ ou 1110____ → Il commence toujours pas 2 bits (2 octets) ou 3 bits (3 octets)

ex : 'é'

Conversion en hexa → binaire

C3 → (110 0011)₂

A9 → (10 01001)₂

Si le prochain de C3 est pas 10, il y a un problème

ASCII Code 233 → 'é'

id du caractère dans l'Unicode

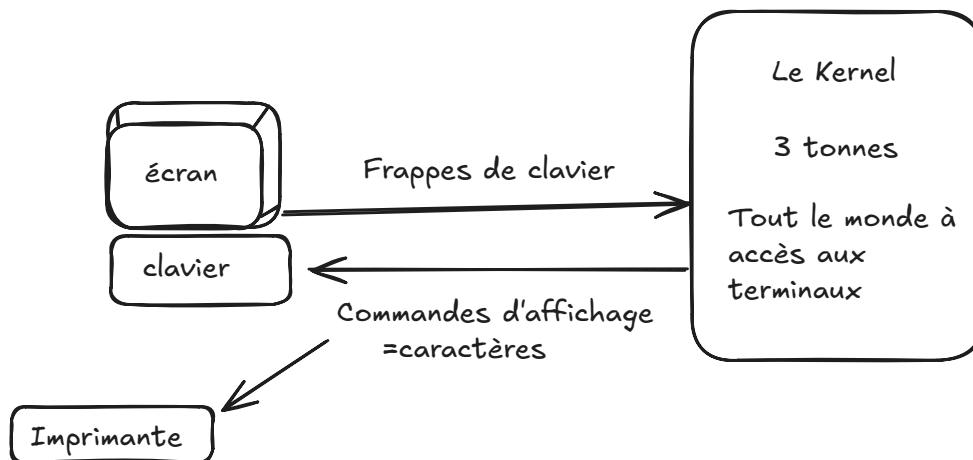
UTF-32 (pas dans le programme)→ 4 octet par caractère

- prends trop de place

On va rester sur les 127 premiers caractères donc même si on ne fait pas de l'UTF-8, on en fait quand même un peu

Avec le C, on ne manipule que la RAM et donc que des nombres

Terminal



Avant les écrans, il y avait des cartes perforées

Pour faire des retours à la ligne, il fallait bouger les chariots de la machine à écrire et les OS Linux utilise 'LF'

`cat/proc/cpuinfo` → montre toutes les extensions utilisé par son CPU

Nombres

1 nombre - 1 octet (au tableau)

1 nombre - 8 octets (sur votre machine)

Nombre **positifs** : 0 jusqu'à $1.8446744e+19$ ($2^{64}-1$)

- **signés** : 1 bit pour le signe, 63 bits pour la valeur
- **virgule** :
 - 1 bit de signe
 - 52 bits de mantisse
 - 11 bits d'exposant

L'exposant est biaisé, la valeur du milieu est 0 est avant c'est négatif et après les positif

Notre premier programme (écrit au tableau)

Tableau avec numérotation des cases puis tableau de programme

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19
20	21	22	23
24	25	26	27
28	29	30	31

-	-	-	-
-	-	-	-
(POP) 2	1	(POP) 2	2
(ADD) 1	MOV	0x30	R2
ADD	PUSH	1	MOV
2	1	MOV	1
2	MOV	1	4
MOVRR	11	3	0x39

CPU

Registres

- ex : nom → id de la case
- IP (instruction pointeur) → 13
- SP → 28
- R1 (Registre 1) → 9
- R2 → 5
- R3
- R4
- *FLAGS*
 - Overflow

Instructions

nom | id | instruction

ADD | 1 | $r1 = r1 + r2$

POP | 2 | arg1 : id registre

SYSCALL | 3 | X

MOV | 4 | Valeur immédiate & Registre

PUSH | 5 | arg1 : id registre

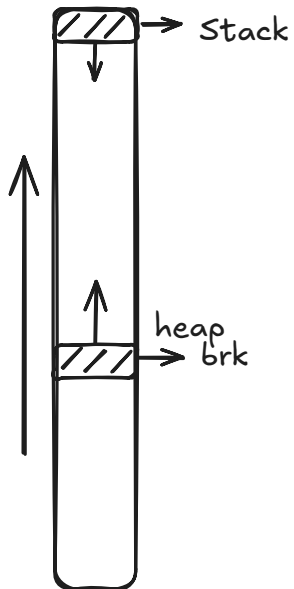
MOVRR | 6 | src (source) → dst
(destination?)

On veut mettre stack et IP dans la RAM

La heap grandie vers le haut et le stack, lui, va descendre

Monter le `brk` pour avoir plus de heap

Un morceau de stack va être attribuer a ta fonction et va être libérer quand un return va être utiliser

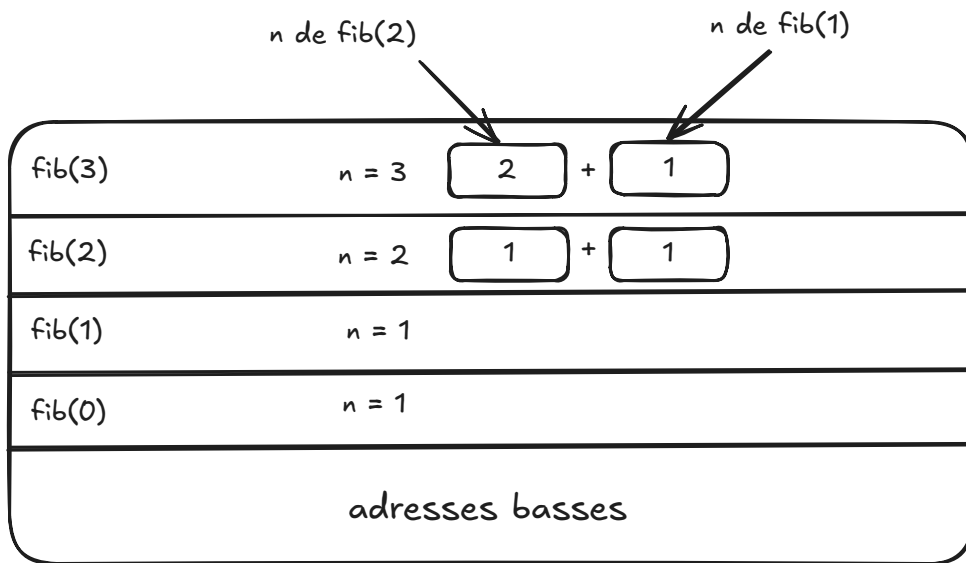


Suite de Fibonacci (en python en premier car on ne connais pas le C)

fib(0)	fib(1)	fib(2)	fib(3)	fib(4)	fib(5)
1	1	2	3	5	8

$\text{fib}(4) = \text{fib}(2) + \text{fib}(3)$

```
def fib(n):  
    if n < 2:  
        return 1  
    return fib(n-1) + fib(n-2)
```



Les Syscalls

Commandes\Registres	R1	R2	R3	R4
exit	1	code d'erreur 0 → OK 1 → KO	X	X
write	2	1 → std (sortie d'erreur) 3 → erreur	numéro de la case marqueur qui contient la 1ère lettre	Le nombre de caractères

Pour grandir le stack, il faut réduire son pointeur

```
fib
sub rsp, 3 ; Réduit le pointeur de stack de 3
```

- **rbp** : Base Pointer (tête de la pile)
- **rsp** : Stack Pointer (fin actuelle de la pile)

ajouter aux instructions - sub | 7 | r1 = r1 - r2

```
fib
sub rsp, 3 ; Réduit le pointeur de stack de 3
```

leave, (pop rsp) pour restaurer le pointeur d'exécution

Mise en place → **Prologue**

```
fib
push rsp
mov rsp, rbp, rsp = rbp
sub rsp 3
```

Instruction à appeler

ret → restaure IP

Paramètre → RDI

Retour → RAX

Appel fib(3)

mov rdi, 3

call fib

cmp → compare la valeur de gauche avec la valeur de droite

fin du code nommée **Epilogue**

Le programme en entier du prof de **Fibonacci**

```
global main

fib:
    push rbp
    mov rbp, rsp
    cmp rdi, 1
    je .fib_ret_1      ;Jump If Equal
    cmp rdi, 2
    je .fib_ret_1

    push rdi           ;Sauvegarde la valeur du registre RDI dans la stack
    sub rdi, 1
    call fib           ;rax = fib(rdi-1)
    pop rdi            ;Restaurer depuis la stack le paramètre de fib

    push rax           ;Sauvegarde le résultat de fib(rdi-1) dans la stack

    sub rdi, 2         ;rax = fib(rdi-2)
    call fib

    add rax, [rsp]     ;rsp pointe vers le résultat de fib(rdi-1)
    leave
```

ret ; => les trois dernières lignes -> épilogue

.fib_ret_1:

```
mov rax, 1
leave
ret
```

print_int:

```
push rbp
mov rbp, rsp
```

```
mov rax, rdi
mov rdx, 0
mov rbx, 10
div rbx
cmp rdi, 10
jl .print_int_end
push rdx
mov rdi, rax
call print_int
pop rdx
```

.print_int_end:

```
add rdx, 0x30
push rdx
mov rax, 1 ; syscall write
mov rdi, 1 ; first argument (fd)
mov rsi, rsp ; second argument (buf)
mov rdx, 1 ; third argument (count)
syscall
leave
ret
```

atoi:

```
mov rax, 0
```

.loop:

```
xor rbx, rbx
mov bl, [rdi]
cmp bl, 0
je .return
mov rcx, 10
mul rcx
sub bl, 0x30
add rdi, 1
add rax, rbx
jmp .loop ; rip = adresse de .loop
```

.return:

```
ret
```

main:


```

push rbp
mov rbp, rsp

; rsi : 1er paramètre (argc)
; rdi : 2ème paramètre (argv)
; 0xADRESSE_DU_PREMIER_PARAM 0xADRESSE_DU_SECOND_PARAM
0xADRESSE_DU_TROISIEME
; Le premier paramètre c'est le nom du programme : "fib"
; Le second paramètre (8 octets plus loin) c'est le paramètre donné par
l'utilisateur

mov rdi, [rsi+8] ; rdi = argv[1] -> 2ème paramètre

call atoi
mov rdi, rax
call fib

mov rdi, rax
call print_int

push 0x0000000A
mov rax, 1
mov rdi, 1
mov rsi, rsp
mov rdx, 1
syscall

mov rdi, 0
mov rax, 60
syscall

```

liste des registres → [CPU Registers x86-64 - OSDev Wiki](#)